D05 · IA·흐름도

# promptbench

명령어 트리 · 핵심 흐름 · 예외 흐름 · 계획 상태 전이도

| | |
|---|---|
| 문서 버전 | 1.0 |
| 작성일 | 2026-10-20 |
| 작성자 | 김태민 |
| 문서 상태 | 구현·실측 완료본 |

## 변경 이력

| 버전 | 일자 | 내용 | 작성자 |
|---|---|---|---|
| 1.0 | 2026-10-20 | 최초 작성 | 김태민 |

## 목차


1. 명령어 트리
2. 핵심 흐름 — 전체 실측 순서 · 승인 흐름
3. 예외 흐름 — 예산 초과 · 상한 정지와 재개 · 실패 재시도
4. 계획 상태 전이도


**이 문서의 역할** — promptbench 는 화면이 아니라 **명령**으로 쓰는 CLI 다. 그래서 정보 구조(IA)를 "어떤 명령이 있고, 과금은 어디서만 일어나며, 계획이 어떤 상태를 거치는가"로 그리고, 사람이 실제로 거치는 순서를 흐름도로 보인다. 정책의 이유는 D03, 요구사항은 D02, 장면 모양은 D06 이 맡는다 — 이 문서가 새 규칙을 만들지 않는다.

**표기 원칙** — 흐름 노드의 `CMD-nn` 은 명령, `SN-nn` 은 CLI 화면, `ST-P-nn` 은 계획 상태, `POL-nn` 은 정책이다. 금액은 실측 전에 정한 **목표치**이고 실측 값은 D09 에 있다. Batch 요청에서 effort 와 structured outputs 를 함께 쓸 수 있는지, 결과 `usage` 에 thinking 토큰이 출력으로 잡히는지 같은 API 동작은 구현 1단계에서 **확인함**(D08 6장)이며, 이 문서의 흐름은 그 결과에 따라 바뀌지 않는다(바뀌는 것은 run 내부 동작뿐).

# 1. 명령어 트리

## 1-1. 명령 7개, 과금은 run 뿐

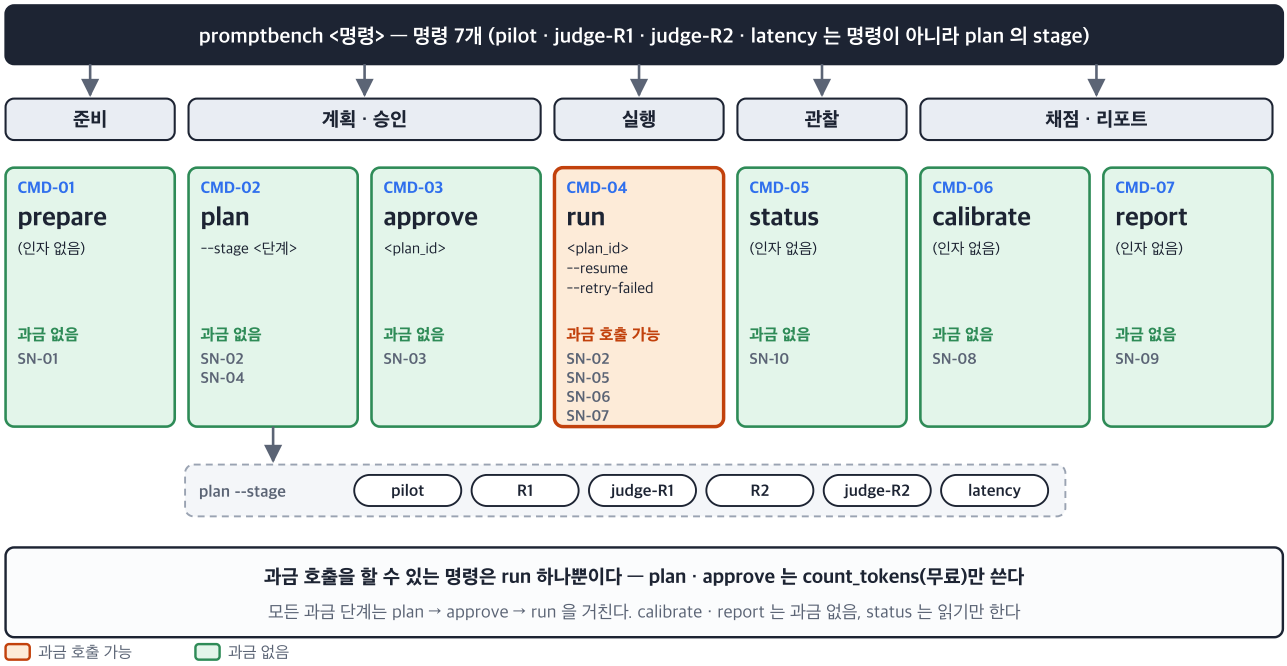



그림 1. 명령어 트리 — 과금 여부와 연결 장면을 함께 표시

- **과금 호출을 할 수 있는 명령은 `run` 하나다.** 승인 전에는 과금 호출이 0건이다 — `count_tokens` 는 무료라 `plan` · `approve` 에서 써도 된다(POL-03).
- **pilot · judge-R1 · judge-R2 · latency 는 명령이 아니다.** `plan --stage` 의 값이다. 단계마다 계획을 하나씩 만들고, 같은 승인 게이트를 거친다.
- `calibrate` 와 `report` 는 계획도 승인도 없다. 과금이 없기 때문이다. `status` 는 장부와 진행 상태를 읽기만 한다.

## 1-2. 명령 일람

| ID | 명령 | 인자 · 옵션 | 과금 | 선행 조건 | 장면 |
|---|---|---|---|---|---|
| CMD-01 | `prepare` | — | 없음 | 없음 (첫 단계 — 데이터 내려받기·표본 분할·표본 ID 기록) | SN-01 |
| CMD-02 | `plan` | `[--stage pilot\|R1\|judge-R1\|R2\|judge-R2\|latency]` | 없음 | prepare 완료. 앞 stage 계획이 ST-P-04 (done) 여야 한다 — R1 은 pilot, judge-R1 은 R1, R2 는 R1 과 judge-R1(T3 이긴 버전을 judge-R1 채점으로 정한다), judge-R2 는 R2, latency 는 R2. ST-P-05 (halted) 는 done 이 아니다 — 앞 stage 가 halted 면 run --resume 으로 먼저 끝내야 한다. 줄이기는 R1(Haiku 비공통 표본) · latency(조합마다 3 → 2 → 1건) 만 하고 pilot · R2 · judge-R1 · judge-R2 는 넘으면 만들지 않는다(POL-21). 만들어진 계획은 ST-P-01 상태로 시작한다 | SN-02<br>SN-04 |
| CMD-03 | `approve` | `<plan_id>` | 없음 | ST-P-01 (draft) 계획. 승인은 계획 해시에 묶인다 | SN-03 |

| ID | 명령 | 인자 · 옵션 | 과금 | 선행 조건 | 장면 |
|---|---|---|---|---|---|
| CMD-04 | `run` | `<plan_id> [--resume] [--retry-failed]` | **과금 호출 가능** | ST-P-02 (approved) 계획. --resume 은 ST-P-05 (halted) 계획, 또는 ST-P-03 (running) 인데 기록된 소유자(잠금 · pid)가 살아 있지 않은 계획 — 받지 않은 배치를 먼저 수집해 장부에 기록한 뒤 남은 예산을 본다. --retry-failed 는 인프라 실패(errored · expired · canceled) · 채점 불가 항목만 담은 새 계획(ST-P-01)을 만들 뿐 실행하지 않는다 (새 계획은 SN-02 로 보인다) | SN-02 SN-05 SN-06 SN-07 |
| CMD-05 | `status` | — | 없음 | 없음 (읽기만 한다) | SN-10 |
| CMD-06 | `calibrate` | — | 없음 | judge-R2 계획이 ST-P-04 (done) — judge-R1 은 R2 의 전제라 이미 done 이다. 교정 세트는 모든 조합의 T3 공통 표본 요약에서 고른다. 사람이 교정 세트 30개를 CLI 에서 채점한다 | SN-08 |
| CMD-07 | `report` | — | 없음 | R1 계획이 ST-P-04 (done) 또는 ST-P-05 (halted). 다른 stage(judge-R1 · R2 · judge-R2 · latency)는 done · halted · 계획 없음 모두 받는다 — halted 이거나 계획이 없는 stage 가 있으면 리포트 맨 위에 '미완료 단계 있음'과 그 stage 목록을 적고, halted 계획은 '계획 일부 실행 — 정지'와 pending 건수를 적는다(POL-08). ST-P-03 (running) 계획이 있으면 거부한다. 기본값 선정은 필요한 stage 가 모두 done 인 업무만 한다(T1 · T2 는 R1 · R2, T3 는 R1 · judge-R1 · R2 · judge-R2) — 나머지 업무는 '선정 보류 — 미완료 단계'로 적는다. 장부와 DB 를 읽어 마크다운과 그림을 만든다 | SN-09 |

`run --retry-failed` 는 인프라 실패(errored · expired · canceled)와 채점 불가만 담은 **새 계획**(ST-P-01)을 만들 뿐 곧바로 실행하지 않는다. 다시 실행하려면 새 계획을 승인해야 한다(POL-10).

### 1-3. plan 의 stage 와 단계별 금액

stage 는 실측 순서대로 `pilot` → `R1` → `judge-R1` → `R2` → `judge-R2` → `latency` 다. R1 은 Haiku(프롬프트 v1 · v2), R2 는 업무별 이긴 프롬프트 하나로 Sonnet · Opus 를 돌린다. 채점(judge)이 둘로 나뉜 것은 R2 의 "이긴 프롬프트 버전"을 업무별로 정하려면 T3 의 R1 품질(채점 결과)이 R2 전에 있어야 하기 때문이다. 채점은 T3 공통 30건 요약만, 채점자는 Sonnet 5.5 · high 다(POL-22). 모든 stage 의 과금은 `run` 에서만 일어난다.

| stage | 계획을 만들 수 있는 조건 |
|---|---|
| pilot | prepare 완료 |
| R1 | pilot 계획이 ST-P-04 (출력 토큰 통계가 있어야 보수 금액을 낸다) |
| judge-R1 | R1 계획이 ST-P-04 |
| R2 | R1 · judge-R1 계획이 모두 ST-P-04 (T3 이긴 버전은 judge-R1 채점으로 정한다) |
| judge-R2 | R2 계획이 ST-P-04 |
| latency | R2 계획이 ST-P-04 (조합이 모두 정해진 뒤) |

ST-P-05 (halted) 는 ST-P-04 가 아니다 — 앞 stage 가 halted 면 `run --resume` 으로 먼저 끝내야 다음 계획을 만든다. 계획이 없는 두 명령의 조건: `calibrate` 는 judge-R2 계획이 ST-P-04(judge-R1 은 R2 의 전제라 이미 끝나 있다), `report` 는 R1 계획이 ST-P-04 또는 ST-P-05 이고 running 계획이 없을 때다. report 는 halted 이거나 계획이 없는 stage 가 있으면 거부하지 않고 리포트 맨 위에 "미완료 단계 있음"과 그 목록을 적으며, 기본값 선정은 필요한 stage 가 모두 ST-P-04 인 업무만 한다(T1 · T2 는 R1 · R2, T3 는 R1 · judge-R1 · R2 · judge-R2).

아래 금액은 손으로 쓴 목표가 아니라 빌드할 때 spec §4 · §4-1 의 배분을 가격표로 계산한 값이다( `tools/budget.py` ). 모두 **목표치**다.

| 순서 | 단계 (plan --stage) | 배분 | 요청 수 | 계산 금액 |
|---|---|---|---|---|
| 1 | pilot | 실제로 쓰는 (모델 × effort) 8개 × 3 업무 × 3건 · v1 | 72 (Haiku 45 · Sonnet 18 · Opus 9) | $0.17 목표치 |
| 2 | R1 | Haiku 5단계 × main 100건 · v1 · v2 | 3,000 (Haiku 3,000) | $0.67 목표치 |
| 3 | judge-R1 | R1 의 T3 공통 30건 요약 채점(Sonnet 5.5 · high) — R2 의 이긴 버전 판정에 필요 | 300 (Sonnet 300) | $1.29 목표치 |
| 4 | R2 | 이긴 버전 하나로 Sonnet low/medium · Opus low × 공통 30건 | 270 (Sonnet 180 · Opus 90) | $1.62 목표치 |
| 5 | judge-R2 | R2 의 T3 공통 30건 요약 채점(Sonnet 5.5 · high) | 90 (Sonnet 90) | $0.39 목표치 |
| 6 | latency | 조합(업무 × 모델 × effort × 버전) 39개 × 공통 앞 3건 · 동기 · 할인 없음 | 117 (Haiku 90 · Sonnet 18 · Opus 9) | $0.36 목표치 |
| — | **합계** | | 3,849 | **$4.51** 목표치 |
| — | 예비 | 실패 재시도 · 채점 불가 재채점 · 추정 오차 — 단계에 배분하지 않음 | | $0.49 목표치 |
| — | 상한 (POL-01) | | | $5.00 — 합계가 안에 든다 |

가정 — 후보 요청 입력 1,000 · 출력(thinking 포함) 700토큰(spec §4-1), 채점 요청 입력 1,800 · 출력 500토큰(Sonnet 5.5 · high — 기사 · 참조 요약 · 후보 요약 · 루브릭을 넣는다고 본 설계 가정). latency 만 동기 호출(할인 없음)이고 나머지는 Batch 50% 할인. 채점 요청 수는 후보 요약이 모두 정상(ok)이라고 본 상한이다. 가격표 config/pricing.yaml(확인일 2026-10-20). 빌드할 때 tools/budget.py 의 budget_lines 를 실행해 얻은 값이며, 파일럿 뒤 계획기가 count_tokens 와 실제 출력 통계로 다시 계산한다. 상한에서 합계를 뺀 나머지는 예비(실패 재시도 · 채점 불가 재채점 · 추정 오차)로 남기고 단계에 배분하지 않는다. 금액은 소수 둘째 자리 반올림이라 단계 금액의 합과 합계가 마지막 자리에서 1 다를 수 있다(1센트).

상한은 실행 한 번이 아니라 프로젝트 전체 누적 장부에 걸린다(POL-01).

## 2. 핵심 흐름

사용자·명령 | 시스템·상태 전이 | **판단** | 끝·상태 | → 다음 단계 · 점선 칸막이 = 갈림길

### 2-1. 전체 실측 순서

사람이 하는 일은 `prepare` 한 번 뒤에 단계마다 **plan** → **approve** → **run** 세 박자를 반복하는 것이다. 사이에 `calibrate` 와 `report` 는 계획 없이 한 번씩 부른다. 실제 순서는 prepare → pilot → R1 → judge-R1 → R2 → judge-R2 → latency → report 이고, calibrate 는 judge-R2 가 끝난 뒤 report 전에 한다.


CMD-01 · SN-01
prepare — 데이터 내려받기 · 표본 분할 · 표본 ID 기록
CMD-02 · SN-02
plan --stage <단계> — 계획 파일 · 해시
ST-P-01
draft
과금 0건
CMD-03 · SN-03
approve <plan_id> — 승인 화면
ST-P-01 → ST-P-02
draft → approved
해시에 승인 기록
CMD-04 · SN-05
run <plan_id> — 배치 제출
ST-P-02 → ST-P-03
approved → running
실행 시작
ST-P-03 → ST-P-04
running → done
모든 배치 종료
① pilot
CMD-02
(모델 × effort) × 3 업무 × 3건 · 프롬프트 v1 — 출력 토큰 평균·p90 기록
② R1
CMD-02
Haiku 5단계 × main 100건 — 프롬프트 v1·v2
③ judge-R1
CMD-02
R1 의 T3 공통 30건 요약을 채점자(Sonnet · high)가 Batch 로 채점 — 이긴 버전 = Haiku 조합 평균 품질이 높은 쪽
④ R2
CMD-02
R1 · judge-R1 완료(done) 필요 — 이긴 프롬프트 1개로 Sonnet low/medium · Opus low × 공통 30건
⑤ judge-R2
CMD-02
R2 의 T3 공통 30건 요약을 채점(Sonnet · high)
⑥ latency
CMD-02
조합 × 공통 앞 3건을 일반 Messages API 로 TTFT 측정 — Batch 할인 없음을 승인 화면에 표시
⑦ calibrate
CMD-06 · SN-08
judge-R2 완료 뒤 사람이 교정 세트 30개를 채점 — 계획 없음 · 과금 없음
⑧ report
CMD-07 · SN-09
집계 · 파레토 그림 · 기본값 선정 — halted · 계획 없는 단계는 "미완료 단계 있음"으로 표시 · 과금 없음
언제든
CMD-05 · SN-10
status — 누적 사용액 · 진행 상태 (읽기만)

- 단계 사이에는 누적 사용액이 $5 안에 있는지 계획기가 매번 다시 확인한다. R2 는 R1 과 judge-R1 이 끝나야(ST-P-04) 계획이 만들어진다 — T3 의 이긴 버전을 judge-R1 채점으로 정하기 때문이다.
- `calibrate` 의 교정 세트 30개는 T3 공통 표본의 요약에서 고정 시드로 골라, 사람이 채점자와 같은 루브릭으로 채점한다.

### 2-2. 승인 흐름

모든 과금 단계는 같은 승인 게이트를 거친다. 승인은 계획 파일의 **해시**에 묶이므로, 계획이 한 글자라도 바뀌면 승인은 무효가 된다 (ST-P-02 → ST-P-01).

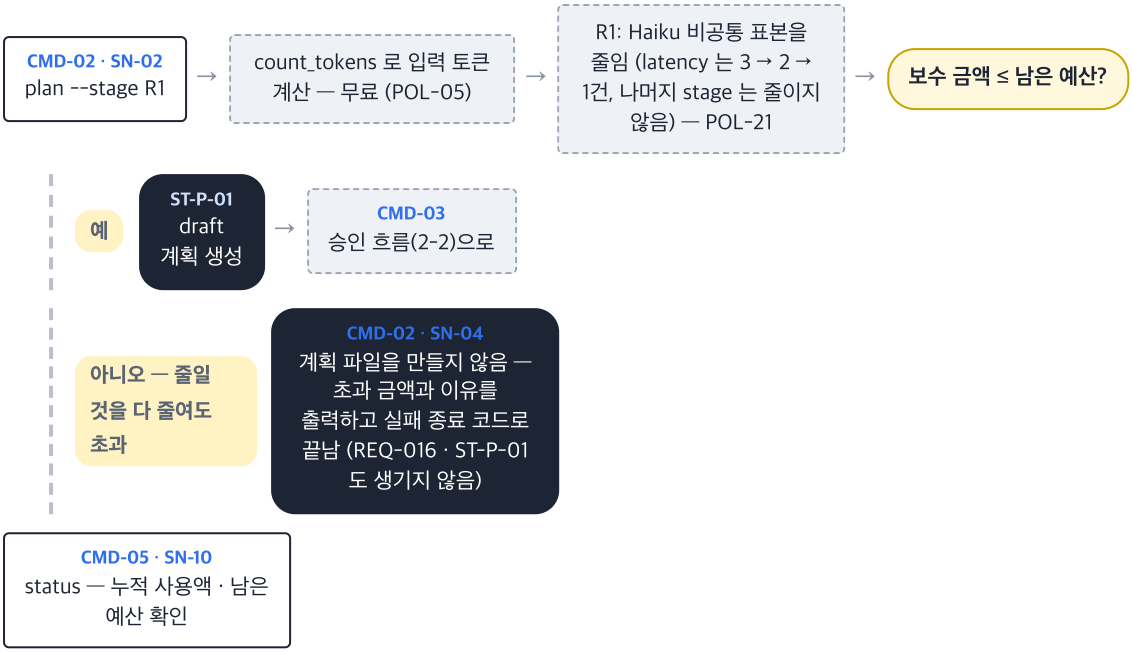



- 승인 화면에는 조합표 · 건수 · 예상/보수/최악 금액 · 누적 사용액과 상한 · 가격표 확인일이 나온다(SN-03).
- 가격표 확인일 경고는 차단이 아니라 경고다. 승인 여부는 사용자가 판단한다.

## 3. 예외 흐름

### 3-1. 예산 초과로 계획 생성 실패

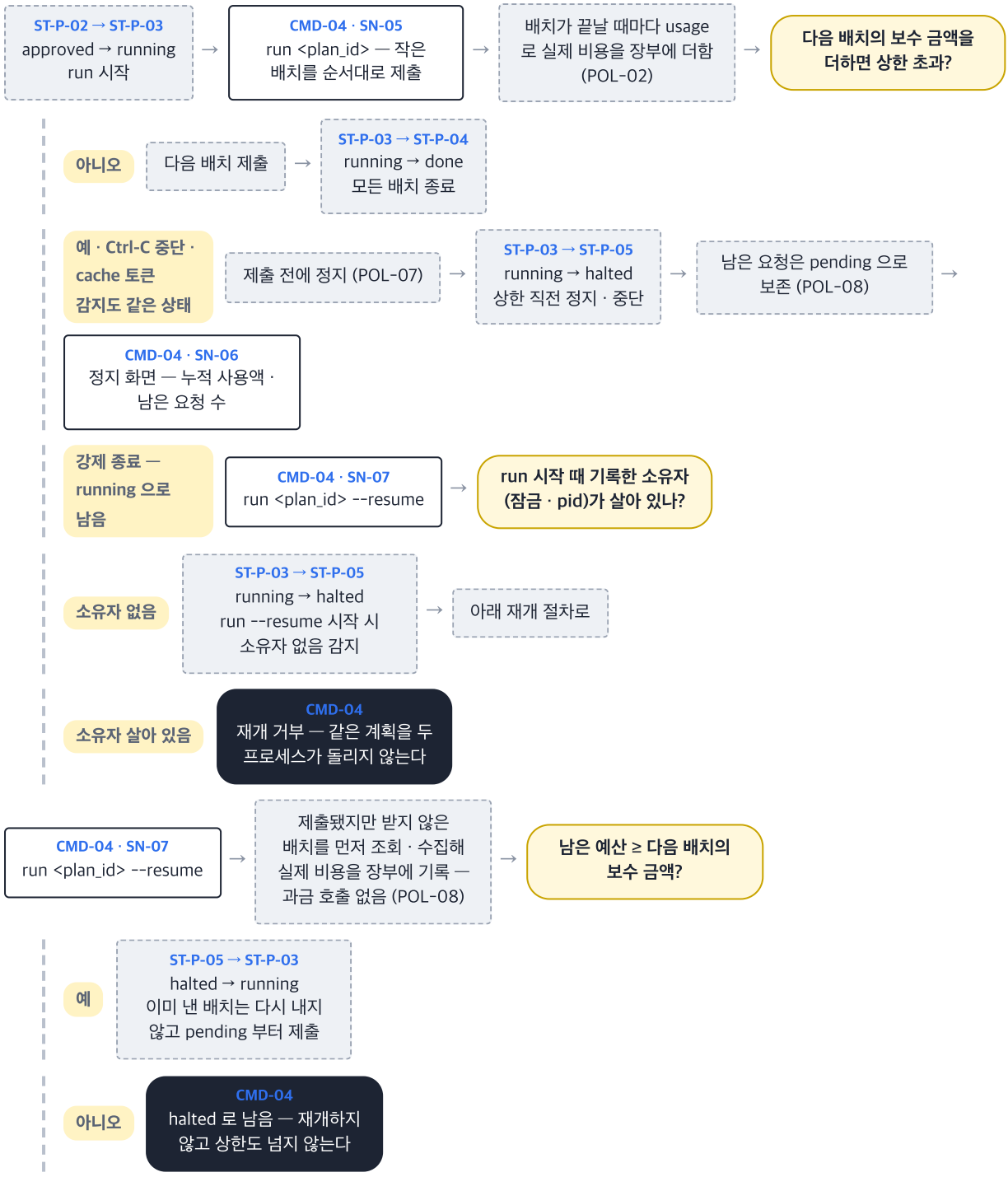
계획기는 보수 금액이 남은 예산 안에 들어올 때까지 줄인다(R1 · latency 만 — 아래). 공통 30건은 줄이지 않으므로, 그래도 넘으면 계획을 만들지 않고 이유를 출력한 뒤 실패(0 아닌) 종료 코드로 끝난다(REQ-016) — 계획 파일이 없으니 승인할 대상도 없고 과금도 없다.




- 줄이는 순서는 stage 마다 다르다(POL-21). **R1**: Haiku 비공통 표본(31~100번째). **latency**: 조합마다 3 → 2 → 1건 (0건으로는 줄이지 않는다). **pilot** 은 줄이지 않는다 — 최악 금액 ≤ 남은 예산이 아니면 계획을 만들지 않는다. **R2** · **judge-R1** · **judge-R2** 도 줄이지 않는다 — 공통 30건 · 이긴 버전 하나가 이미 최소 배분이라 넘으면 계획을 만들지 않는다.

### 3-2. 상한 정지와 재개

상한은 사후 확인이 아니라 **제출 전** 검사다. 다음 배치를 내면 상한을 넘을 것이 보이면 내지 않고 멈추므로 상한을 넘겨 쓰는 일이 없다.




- `run --resume` 이 받는 계획은 둘이다. ① halted(ST-P-05) ② running(ST-P-03)인데 실행 소유자가 살아 있지 않은 계획 — `run` 은 시작할 때 소유자(잠금 · pid)를 계획에 기록하므로, 프로세스가 강제 종료돼 running 으로 남은 계획을 알아볼 수 있다. ② 는 먼저 halted 로 바꾼다(ST-P-03 → ST-P-05). 소유자가 살아 있는 running 계획은 재개하지 않는다(같은 계획을 두 프로세스가 돌리지 않는다).
- 재개는 먼저 **제출됐지만 결과를 받지 않은 배치**를 DB 의 배치 ID 로 조회 · 수집하고 실제 비용을 장부에 기록한다 — 과금 호출이 아니다(이미 낸 배치다). 그 뒤에 남은 예산 ≥ 다음 배치의 보수 금액인지 본다. 순서가 반대면 아직 장부에 없는 비용 때문에 상한 판정이 틀린다.
- 재개는 새 승인을 요구하지 않는다 — 승인된 해시가 그대로이고 같은 계획을 이어 가기 때문이다(spec §3-1 원칙 1 · §7: 이미 낸 배치는 다시 내지 않는다). 남은 예산이 모자라면 halted 로 남는다. 24시간까지 걸릴 수 있는 배치 대기를 끊어도 마찬가지다.

### 3-3. 실패 재시도 — 새 계획

배치 항목이 errored · expired · canceled 로 끝나면 오답으로 집계하고 유형별 건수를 보고한다. 자동 재시도는 없다. 사람이 원하면 인프라 실패(errored · expired · canceled)와 채점 불가만 담은 **새 계획**을 만들고, 그 계획을 처음부터 다시 승인한다.

재시도 대상은 **인프라 실패**(errored · expired · canceled)와 T3 의 **채점 불가**뿐이다(POL-10). 거절(refused) · `max_tokens` 잘림 · 형식 파싱 실패는 모델이 낸 답이므로 확정된 오답이며 재시도하지 않는다 — 다시 보내 다른 답을 얻으면 사전 등록한 비교(POL-13)가 "될 때까지 돌리기"가 된다. 재시도 결과는 실패 결과를 대신해 집계에 들어간다. 후보 재요청은 어느 요청을 대신했는지(`retry_of`)를, 채점 불가의 재채점은 같은 결과의 새 `judgments` 행(`judgment_seq` · `plan_id`)을 남긴다.




- 원래 계획은 done 으로 남는다. 새 계획은 새 해시를 가지므로 이전 승인이 재사용되지 않는다.
- 재시도 결과는 같은 (조합, 항목)의 실패 결과를 대신한다. 실패 결과 행은 지우지 않고, 인프라 실패로 다시 보낸 후보 요청은 새 결과에 `retry_of` = 실패한 요청의 request_id 를 남겨 계보를 추적하고, 채점 불가의 재채점은 같은 결과의 새 `judgments` 행(`judgment_seq` 증가 · 재채점 계획의 `plan_id`)으로 계보가 남는다(D04).

## 4. 계획 상태 전이도

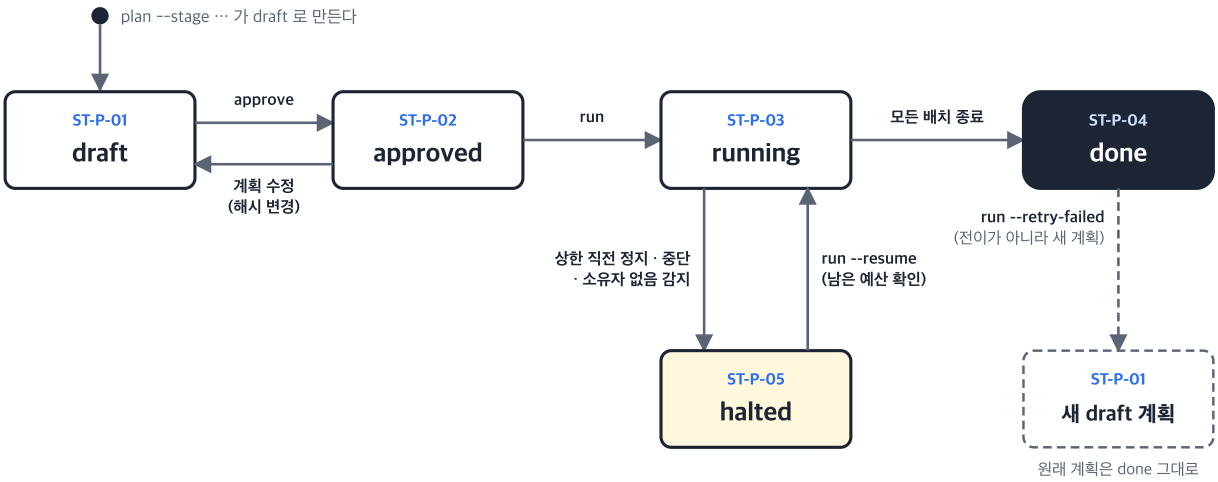



그림 2. 계획 상태 전이도 — 화살표 라벨은 일으키는 명령·사건

| ID | 상태 | 뜻 |
|---|---|---|
| ST-P-01 | `draft` | 계획 파일이 만들어졌지만 아직 승인되지 않은 상태. 과금 호출은 0건이며 count_tokens 만 쓴다. |
| ST-P-02 | `approved` | 계획 해시에 승인이 기록된 상태. 이 상태의 계획만 run 으로 실행할 수 있다. |
| ST-P-03 | `running` | 배치를 작게 나눠 순서대로 제출하며 배치마다 실제 비용을 장부에 더하는 상태. run 은 시작할 때 소유자(잠금 · pid)를 계획에 기록하고 끝나면 지운다. |
| ST-P-04 | `done` | 모든 배치가 끝난 상태. 나가는 전이가 없다 — 실패 재시도는 이 계획을 되살리지 않고 새 draft 계획을 만든다. |
| ST-P-05 | `halted` | 상한 직전에 멈추거나 프로세스가 중단된 상태. 이미 제출한 배치는 다시 내지 않고 남은 요청은 pending 이다. 제출됐지만 결과를 받지 않은 배치가 있을 수 있다. |

### 4-1. 전이 표

| 출발 | 도착 | 일으키는 명령·사건 | 조건(guard) |
|---|---|---|---|
| ST-P-01<br>draft | ST-P-02<br>approved | `approve <plan_id>` | 승인 화면에서 y 를 입력하고, 승인 시점의 계획 해시가 계획 파일의 해시와 일치 |
| ST-P-02<br>approved | ST-P-03<br>running | `run <plan_id>` | 승인된 해시와 계획 파일의 해시가 같음 |
| ST-P-02<br>approved | ST-P-01<br>draft | `계획 수정` | 계획 내용이 바뀌어 해시가 달라짐 — 승인이 무효가 되어 다시 승인해야 함 |
| ST-P-03<br>running | ST-P-04<br>done | `모든 배치 종료` | 계획의 모든 요청이 성공 또는 실패(errored·expired·canceled)로 확정됨 |

| 출발 | 도착 | 일으키는 명령·사건 | 조건(guard) |
|---|---|---|---|
| ST-P-03 running | ST-P-05 halted | `상한 직전 정지 · 중단` | 다음 배치의 보수 금액을 더하면 $5 상한을 넘거나, 수집한 결과에 cache 토큰이 0 이 아니거나(비용 확정 불가 — 장부에는 기록하고 멈춘다, POL-02), 사용자가 중단(Ctrl-C)해 실행 중인 프로세스가 halted 를 기록함 — 남은 요청은 pending 으로 보존 |
| ST-P-03 running | ST-P-05 halted | `run --resume 시작 시 소유자 없음 감지` | 계획이 running 인데 run 시작 때 기록한 소유자(잠금 · pid)가 살아 있지 않음 — 강제 종료로 halted 를 기록하지 못한 계획. 소유자가 살아 있으면 전이하지 않고 재개를 거부한다 |
| ST-P-05 halted | ST-P-03 running | `run <plan_id> --resume` | 먼저 제출됐지만 결과를 받지 않은 배치를 DB 의 배치 ID 로 조회 · 수집해 실제 비용을 장부에 기록한 뒤(과금 호출 없음), 남은 예산이 다음 배치의 보수 금액 이상 |

- `done` 만 나가는 전이가 없다. `halted` 는 `run --resume` 으로 `running` 에 돌아갈 수 있다.
- `running → halted` 는 두 경우다. 실행 중인 프로세스가 상한 직전 정지 · 중단(Ctrl-C)을 기록하는 경우와, 강제 종료로 running 에 남은 계획을 `run --resume` 이 시작할 때 소유자 없음으로 감지하는 경우다.
- `approved → draft` 는 사용자 명령이 아니라 계획 수정의 결과다. 계획 파일의 해시가 달라지면 승인이 무효가 된다.
- `run --retry-failed` 는 이 전이도 안의 화살표가 아니라 새 계획의 생성이다.